OS Lab v3.0 — Developer Manual

Architecture, implementation, testing, release and maintenance handbook

OS Lab Project

27 October 2026

Contents

1. Purpose and engineering scope **1**

2. Provenance and evolution **2**

3. Actual repository structure **2**

4. Application entry point and rendering model **3**
Extension chain and why v2.0 initially failed . . . 3

5. Course engine and learning method **3**

6. Legacy simulation layer **4**

7. Engineering Lab: runtime lifecycle **4**
Virtual machine fixture . . . 5
Scenario injection and reset . . . 5

8. CLI design: supported subset and threat boundaries **6**
Source examples and their meaning . . . 6

9. Scoring, assessment, analytics and persistence **6**
Browser storage keys and backup semantics . . . 7

10. Security and offline requirements **7**

11. Local development and debugging **7**
Targeted acceptance protocol . . . 8

12. Safe maintenance procedure **8**

13. Future architecture (proposed, NOT currently implemented) **8**

14. GitHub development and release **9**
New empty repo — essential commands . . . 9
Optional release tag . . . 9

15. Baseline integrity and reproducibility **9**

16. Maintainer handover checklist **10**

1. Purpose and engineering scope

This manual documents the **existing user-tested OS Lab v3.0 engineering build**, preserving the v2.0.1 Stage 1–8 application and its additional Engineering Lab extension. It is intended for a new maintainer who needs to reconstruct the source tree, understand the integration strategy, reproduce the offline release, find and fix defects, and publish updates responsibly.

Baseline integrity rule: The documentation/repository package intentionally leaves the functional HTML/CSS/JavaScript source unchanged from the supplied v3 build. The manuals and verification scripts are additive. Developer references to `src/`, modular state managers or future state unification in earlier plans should not be mistaken for files that already exist in this release.

This is **not** a bootable kernel, hypervisor, Docker replacement, or full implementation of Bash/PowerShell/CMD. Its educational OS models operate inside JavaScript in the browser. Workloads, users, firmware entries, routes, logs and files in the simulator are fictional. There is no real host shell execution.

2. Provenance and evolution

Milestone	Architectural contribution	Evidence available
Stage 1 / v1.0	First offline page, Module 1, introductory terminal, desktop, faults, local saved progress	Previous ZIP retained separately by owner
Stages 2–6	Modules 2–10, boot/kernel, scheduling, memory, file/device/network/security simulations	User reported earlier acceptance
Stages 7–8 / v2.0	Modules 11–12, VM allocator, mini-OS, 60 total scenarios	Initial Module 11 placeholder defect observed
v2.0.1	Fixed Stage 7 lesson routing, Stage 8 scenario verification/reset	User confirmed corrected build passed
v3.0 Engineering Build	Added `engineering.js`, three shell modes, category-level shared virtual state, assessment history	User reported tests passed and supplied Ubuntu screenshots
Documentation release	Adds full manuals, GitHub publishing guide, verification script and screenshots	**No functional JS changes** in this package

The project owner's interactive confirmation demonstrates the tested workflows on Ubuntu, **not** exhaustive cross-platform validation or proof of every edge case. The original engineering notes remain in `ENGINEERING_NOTES.md` to preserve stated limitations.

3. Actual repository structure

```
os-lab/
├── index.html              # Page shell and ordered script tags
├── style.css               # Shared styling, navigation, desktop/terminal panels
├── app.js                  # Stage 1 base application and mutable globals
├── stages2-6.js            # Course extension, original simulations and scenarios
├── stages7-8.js            # Additional lessons, mini-OS, guest allocator, cases 17–60
├── engineering.js          # v3 independent engineering machine and assessment UI
├── README.md               # GitHub landing and setup instructions
├── ENGINEERING_NOTES.md    # Original v3 scope/limitations (unchanged)
├── FIX_NOTES.md            # v2.0.1 integration defect history (unchanged)
├── docs/
│   ├── USER_MANUAL.md
│   ├── DEVELOPER_MANUAL.md
│   ├── GITHUB_PUBLISHING.md
│   ├── RELEASE_CHECKLIST.md
│   └── assets/
│       ├── engineering-lab.png
```

```
│       └── virtual-desktop.png
└── scripts/
    └── verify_release.py   # Non-invasive structural/syntax checks
```

No package manager is required at runtime. `index.html` loads local scripts in this exact order:

```
<script src="app.js"></script>
<script src="stages2-6.js"></script>
<script src="stages7-8.js"></script>
<script src="engineering.js"></script>
```

The script order is an architectural dependency. These are classic scripts sharing a browser global lexical environment; newer releases override existing functions or append to shared arrays after the older ones run. Do **not** reorder or add `type="module"` indiscriminately without refactoring all dependent symbols.

4. Application entry point and rendering model

`index.html` contains the navigation buttons, sidebar, `#modules` and `#content` containers. `app.js` initializes base state and provides globals such as:

- `names`: array of the 12 module labels (later stages can replace labels).
- `state`: course progress loaded from browser storage.
- `view`: selected view (`learn`, `desktop`, `lab`, `progress`, then `engineering`).
- `activeModule`: zero-based module index.
- `scenarios`: mutable array of original fault descriptions; later stages append cases.
- `render()`, `renderModules()`, `learn()`, `desktop()`, `lab()`, `progress()`.
- `command(raw)`: original simulated CLI handler.
- `wireTerm()` and related DOM event wiring.

A common operation is `render()` setting `#content.innerHTML` based on `view`, then attaching handlers. A new view can therefore be lost unless its wrapper intercepts the correct `view` value. Dynamic DOM replacement also removes old event listeners and destroys transient input state: initialize or rewire intentionally after redraws.

Extension chain and why v2.0 initially failed

1. Base `learn()` in `app.js` displayed a Module 1 lesson and placeholders for others.
2. `stages2-6.js` replaced `learn()` with detailed Module 2–10 rendering and left Modules 11–12 as placeholders.
3. The original v2.0 Stage 7 extension updated module navigation but did **not** update lesson routing; Modules 11–12 still rendered the placeholder.
4. The v2.0.1 fix added a `learn()` wrapper routing indices 10–11 through the stage course renderer, plus Stage 8 scenario state corrections.
5. `engineering.js` then added a new navigation button and a separate `view==='engineering'` render path.

Regression rule: Every future extension must verify *both* navigation labels **and** actual lesson content, event handlers, simulator state, reset behavior, and exported progress. A passing syntax check does not confirm browser behavior.

5. Course engine and learning method

Module 1 content is in `app.js`. The later courses are represented in the `COURSE` array in `stages2-6.js` and appended/extended by `stages7-8.js`. `courseLesson(i)` assembles the nine-step page:

1. Topic and objectives.
2. Definitions.
3. Step-by-step explanations.
4. Interactive diagrams.

5. Commands and examples.
6. Real-world applications.
7. Exercises and quizzes.
8. Topic mini-project.
9. Integrated project contribution.

The interactive simulation panel is selected by `simUI(i)` and wired through stage-specific handler code. Module indices are **zero-based internally** and **one-based in the UI**. For example, UI Module 11 corresponds to `activeModule===10`.

The user-facing syllabus: fundamentals; boot/GRUB; kernel architecture; processes/scheduling; concurrency; memory; filesystems/storage; devices/I/O; networking; security/administration; virtualization/containers; mini-OS.

A lesson is marked complete when the user clicks its completion button. The quiz records a score but does not necessarily enforce competency thresholds or project completion. Do not label completion as externally verified mastery.

6. Legacy simulation layer

The original virtual desktop includes a simulated Terminal, Files, System Monitor and Settings. It uses the base `command()` or stage wrappers and their original state objects; the original Troubleshooting page uses the `scenarios` array and per-scenario fault flags.

Stage 7 introduces:

- Kernel architecture comparison.
- Virtual guest resource allocator (teaching model, not real virtual machines).
- Mini-OS workbench (`oslab mini boot|run|file|network|secure|health|reset`).

Stage 8 appends 44 cases to a total of 60. The stage-specific terminal supports e.g. `oslab inspect case17` and `oslab repair case17`, and verification relies on the matching virtual fault flag. These are **purpose-built teaching commands**.

Isolation boundary: The legacy desktop and the newer Engineering Lab do *not* use one globally unified virtual machine state. Changing network configuration in the Engineering Lab must not be documented as changing the classic desktop or legacy Troubleshooting view. The common UI shell and navigation do not imply common simulation state.

7. Engineering Lab: runtime lifecycle

`engineering.js` is an immediately invoked function expression (IIFE) and implements the v3 add-on. Its major components are:

```
load engineering.js
    |
    +--> read engineering assessment localStorage
    |
    +--> baseline() : new virtual machine fixture
    |
    +--> faultCases / scenarios : 60 selectable prompts
    |
    +--> applyFault(selectedScenario)
    |
    +--> add Engineering Lab nav button
    |
    +--> when view === engineering: paint()
              |
              +--> render scenario selector, shell, mode
              +--> render virtual terminal and score panels
              +--> execute command(raw) against local vm
```

```
            +--> run verify / diagnosis / submit
            +--> persist attempts and best scores
```

Key functions in the source include:

Function	Responsibility
`baseline(os)`	Construct a fresh fictional Linux/Windows-like VM fixture
`applyFault(i)`	Inject a fault based on the selected case/category
`isFixed()`	Evaluate whether the fault-related virtual state is repaired
`command(input)`	Parse a bounded safe subset of simulated CLI operations
`start(i, m)`	Reset the VM and session for scenario/mode
`score()`	Compute approximate evidence-based assessment points
`submit()`	Record a scored attempt in local storage
`section()`	Build Engineering Lab UI HTML
`paint()`	Render and wire controls; update status/output
`persist()`	Save assessment history (independent of course progress)

The v3 extension also exposes `window.OSLabEngineeringTest` with `baseline`, `applyFaultForTest`, `run`, `check`, `score`, `scenarioCount`, `getState`, and `reset`. It is a **development test hook** operating within the browser. Avoid using it as a security boundary or encouraging users to modify assessment scores through the console.

Virtual machine fixture

A new `baseline()` machine includes:

```
vm.os, host, user, admin
vm.network: ip, prefix, gateway, dns, up, mtu
vm.services: web, nginx, ssh, Spooler
vm.processes: pid, name, ram, cpu
vm.memory: total, used, swap
vm.storage: total, used, mounted, readonly
vm.files: path -> {data, owner, mode}
vm.boot: entry, root, initramfs
vm.devices: ethernet, bluetooth, wifi
vm.logs, vm.fault, vm.fixes, vm.ticks
```

This structure is an educational in-memory JavaScript object, not a filesystem image. Storage units are labeled GiB; memory is labeled MiB, and some counters are schematic rather than a complete accounting model.

Scenario injection and reset

`start()` constructs a fresh baseline, calls `applyFault()`, clears command/assessment trace, and resets the session timer. Scenario selection or mode/shell selection also restarts. Do not write tests expecting fault state to persist across reload or shell switch.

Most Engineering Lab cases map to a **category-level state change**, based on `scenarios[i].category` and text. Examples: bad DNS, missing gateway, interface down, web stopped, nearly full/read-only storage, memory pressure, runaway process, missing boot entry, and file permission denial. Different case descriptions can lead to the same fault mechanism. This is explicitly a **limitation**, not a claim that all 60 cases have unique realistic roots.

8. CLI design: supported subset and threat boundaries

The Engineering Lab interpreter:

1. Trims the supplied text and caps commands at 300 characters.
2. Rejects shell operators `;`, `&`, `|`, backticks and `$` (simplified safety policy).
3. Splits on whitespace; **it does not support full quoting/pipelines/redirection**.
4. Dispatches recognized tokens to internal JavaScript branches.
5. Reads/mutates the in-memory `vm` object.
6. Records some observations/state changes in `trace` and changes in `vm.logs`.
7. Renders the returned string in a text node (`textContent`) in the virtual terminal.

No real `child_process`, shell, network API or device access is invoked. A browser's normal local HTML loading is the deployment mechanism.

Source examples and their meaning

Command	Educational model behavior
`ip addr` / `ipconfig` / `Get-NetIPConfiguration`	Show virtual interface/IP/gateway/DNS
`ip route` / `route print` / `Get-NetRoute`	Show virtual default route
`resolvectl status`	Show configured virtual DNS
`resolvectl dns eth0 1.1.1.1`	Admin-only update of virtual DNS setting
`systemctl status web` / `Get-Service`	Show modeled service status
`ps` / `tasklist` / `Get-Process`	Show modeled process rows
`free -h`	Display modeled memory counters
`df -h`	Display modeled storage counters
`diagnosis <explanation>`	Record a keyword-recognized diagnosis event
`verify`	Check whether the modeled fault has been resolved
`fix storage` etc.	Educational shortcut for category faults not fully covered by native-like commands

Command semantics caveat: The parser does not fully enforce real OS-specific argument grammar. Some actions (file operations, service commands) are simplified and lack production OS authorization behavior. The UI advertises "simulated" prominently. Do not relabel the terminal as a real shell.

9. Scoring, assessment, analytics and persistence

`score()` in the v3 extension awards:

Criterion	Max points	Current implementation
Observations	20	Up to four counted observations at 5 points each
Diagnosis	25	Awarded when a diagnosis event is recorded; not robust semantic checking
Repair	25	Requires a change event **and** fixed state
Verification	20	Requires a successful recorded `verify`
Explanation/reconciliation	10	Awarded when diagnosis is recorded and fault is fixed
Total	**100**	Self-study rubric; not proctored certification

Passing condition in `submit()` is **score ≥ 80, repaired, and verified**. Attempt records include scenario ID, title, assessment mode, points, pass flag, ISO timestamp and action count. The last 300 attempts are retained in the Engineering Lab history (with per-case best score also stored).

Known grading concern: The recorded-diagnosis score can be easier to obtain than genuine root-cause understanding; the UI's "evidence-based" description should be understood as approximate. Future work should record accepted diagnostic evidence and validate semantic diagnosis if assessment rigor is required.

Browser storage keys and backup semantics

Key	Origin	Content	Backup surface
`oslab-stage1-v1`	Original `app.js`	Completed module indices, solved scenario indices, quiz results and stage scores	**Progress → Export JSON** (import supported)
`oslab-v3-engineering-v1`	`engineering.js`	Schema 1, attempts, best scores, history	**Engineering Lab → Export assessment JSON** (import **not** implemented)

The progress JSON exported from the original UI reports `version: 3` and can import older course exports. Engineering Lab exports its separate store. Moving `index.html`, changing profile/browser, using private mode, or clearing local storage may affect data. Avoid running migration logic without representative sample exports.

10. Security and offline requirements

- **Never** add `eval`, `new Function`, `child_process`, local execution adapters or an unrestricted command runner.
- Retain `textContent` for displaying arbitrary command inputs/outputs and escape dynamic strings inserted into HTML.
- Avoid remote scripts/styles/fonts. The runtime must load fully without Internet.
- If adding import features, validate JSON structure, bounded arrays, known keys, version and maximum size. In particular, the current generic progress importer should not be treated as hardened against adversarially large files.
- Do not record actual user passwords or keys as part of learning assessments.
- Limit log/trace growth to avoid runaway DOM or memory usage.
- Respect keyboard navigation and maintain readable contrast on the dark UI.

Security review should include hostile strings, malformed JSON, oversized input, and keyboard-only navigation. This documentation does not assert that a formal security audit has been completed.

11. Local development and debugging

The app can be launched without a server:

```
cd path/to/os-lab
xdg-open index.html
```

From Firefox, use **F12 → Console** to inspect JavaScript errors. Keep the source files together. Developer tools are available even offline.

Node.js is helpful for static syntax checking, but not needed for runtime use:

```
node --check app.js
node --check stages2-6.js
```

```
node --check stages7-8.js
node --check engineering.js
```

Run the additive repository verifier (Python 3 is a **developer dependency only**):

```
python3 scripts/verify_release.py
```

It verifies source-file presence, the script tag order, the expected course and case declarations, local references, and syntax if Node is available. It **does not** open a browser or prove the app's behavioral semantics.

Targeted acceptance protocol

1. **Core UI:** Launch locally; verify all 12 module buttons render real lesson sections, especially Module 11.
2. **Legacy desktop:** Open Terminal, Files, System Monitor, Settings; try `help`, `ls`, `free -h`.
3. **Stage 7:** Allocate a guest; verify host-capacity limits; boot/run/secure the mini-OS; request health.
4. **Stage 8:** In classic Troubleshooting, choose case17; inspect, repair, verify, reset.
5. **Engineering:** Choose DNS scenario, Bash mode; inspect using separate commands; repair DNS; verify and submit.
6. **Cross-shell:** Switch to CMD/PowerShell and inspect the virtual network/process inventory.
7. **Course backup:** Export course JSON, complete a new action, import and verify relevant fields.
8. **Assessment export:** Submit a score, refresh browser, confirm analytics and export assessment JSON.
9. **Offline:** Disconnect from the network and repeat representative navigation and CLI actions.
10. **Browser:** Verify Firefox on Ubuntu; Windows browser acceptance is a separate manual gate and is not claimed here.

The owner reported their acceptance tests passed on Ubuntu. Include any subsequent reproducible defects in the Git issue tracker with build version, browser, scenario, exact commands, expected/actual output and screenshots if useful.

12. Safe maintenance procedure

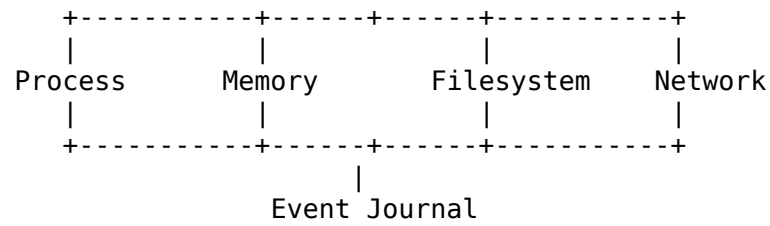
1. Export coursework JSON and Engineering Lab assessments **separately**.
2. Copy the last known working repository/archive to a new release directory.
3. Make one focused change on a feature branch; do not edit the accepted version in place.
4. Run syntax and structural checks.
5. Test direct file opening in a browser (not only a dev-server origin).
6. Recheck Modules 11 and 12 and case17, as these were historical regression points.
7. Test restoration of a pre-change course progress export.
8. Run the manual acceptance checklist in `docs/RELEASE_CHECKLIST.md`.
9. Review `git diff` to ensure unrelated application files were not modified.
10. Commit with a descriptive message and push to a repository chosen by the owner.

For a documentation-only release, app source checksums should stay identical to the provided v3 baseline. The release packager can validate this mechanically (see section 15).

13. Future architecture (proposed, NOT currently implemented)

When development resumes, a fully unified state model would separate UI from simulation and make all views access one store:

```
Course / Desktop / Terminal / Troubleshooting / Assessments
                        |
                  Action Dispatcher
                        |
                  Virtual Machine State
```



Such a change requires careful compatibility and re-testing. Do **not** claim it already exists in this version. Likewise, a future development backlog may include command tokenizer/quoting, individually unique fault fixtures, more detailed resource timelines, assessment-history import, stricter root-cause evaluation and browser automation.

14. GitHub development and release

Publishing from the user's Ubuntu workstation is the most direct method when GitHub is not connected as a writable service. Full step-by-step workflows for **new and existing repositories** are provided in `docs/GITHUB_PUBLISHING.md`.

New empty repo — essential commands

After downloading/extracting the documentation release and creating a matching **empty** GitHub repo (without autogenerated README/license), inside the folder containing `index.html`:

```
git init -b main
git add .
git diff --cached --stat
git commit -m "docs: finalize OS Lab v3 user and developer manuals"
git remote add origin <REPOSITORY_URL>
git push -u origin main
```

<REPOSITORY_URL> must be replaced by the actual GitHub URL. If this directory is **already** a repository, do not reinitialize or blindly overwrite the `origin` remote. See the publishing guide.

Optional release tag

After verifying the commit and after GitHub reflects the push:

```
git tag -a v3.0.0-docs -m "OS Lab v3.0 user-accepted engineering build documentation"
git push origin v3.0.0-docs
```

The tag records a release/documentation milestone, not full production-level semantic completeness.

15. Baseline integrity and reproducibility

The source archive supplied for this documentation pack is `OS_Lab_v3.0_Engineering_Phases_0-6.zip`. The packager extracts the nine original files into `os-lab/` and **adds** documentation, screenshots, `.gitignore`, and a verifier. No original JavaScript, CSS or HTML runtime file is edited. The original `ENGINEERING_NOTES.md` and `FIX_NOTES.md` remain as provenance.

To audit post-extraction file integrity against the original ZIP, compare SHA-256 digests of:

```
app.js
style.css
index.html
stages2-6.js
stages7-8.js
engineering.js
ENGINEERING_NOTES.md
FIX_NOTES.md
```

A manifest of these baseline hashes is included in `docs/BASELINE_SHA256.txt`. This manifest is an integrity reference, not an assertion that the program is defect-free. The verifier compares the extracted files with the recorded values.

16. Maintainer handover checklist

- ☐ Clone/download the complete repository and understand script load order.
- ☐ Open the app with `file://` on Ubuntu and inspect the Console.
- ☐ Identify the two independent terminal/state systems.
- ☐ Reproduce a legacy case17 fault and an Engineering DNS fault.
- ☐ Locate the course and Engineering Lab storage keys.
- ☐ Export coursework and assessment history separately.
- ☐ Run `scripts/verify_release.py`; record any static warning.
- ☐ Review the 60-case modeling limitation and the simplified assessment rubric.
- ☐ Preserve release source hashes when making a docs-only change.
- ☐ Publish only after the owner has chosen the GitHub repository and checked `git status`.